

II Parte: descrizione informale

- ▶ Programmazione Intera
- ▶ Ottimizzazione Combinatoria e Programmazione Intera
- ▶ Esempi di formulazioni tramite programmi interi
- ▶ Qualità delle formulazioni
- ▶ Stime e qualità delle soluzioni:
 - ▶ Rilassamenti
 - ▶ Dualità
- ▶ Dualità Lagrangiana
- ▶ Tecniche di base per la soluzione di PI e di OC:
 - ▶ Algoritmi di Branch and Bound
 - ▶ Dynamic programming (?)
- ▶ Metodi di approssimazione
 - ▶ Algoritmi Euristicici (?)

Dai Programmi Lineari ai Programmi Interi

Un problema di programmazione/ottimizzazione lineare (PL):

- ▶ Un problema di programmazione matematica:
- ▶ Ottimizzare una funzione obiettivo *lineare*...
- ▶ sotto vincoli *lineari* per variabili di decisione...
- ▶ e le variabili di decisione sono **continue**, cioè possono assumere valori reali

Un problema di programmazione/ottimizzazione lineare intera (PI):

- ▶ Un problema di programmazione matematica:
- ▶ Ottimizzare una funzione obiettivo *lineare*...
- ▶ sotto vincoli *lineari* per variabili di decisione...
- ▶ e le variabili di decisione sono **interi**, cioè possono assumere solo valori interi

Programmazione Intera: Che cosa studieremo?

- ▶ Molti problemi della vita reale richiedono variabili intere o decisioni discrete
 - ▶ È necessario estendere il vocabolario di modellizzazione della PL
- ▶ I Metodi per risolvere i PL non funzionano (usualmente) per risolvere i PI
 - ▶ Bisogna sviluppare metodi di soluzione (algoritmi) per i PI
- ▶ Risolvere i PI richiede una grande quantità di memoria e tempo
 - ▶ Capire ciò che rende i PI difficili da risolvere
 - ▶ Identificare quei PI che hanno qualche caratteristica speciale
 - ▶ Sviluppare metodi di soluzione specifici che sfruttano queste caratteristiche per risparmiare memoria e/o tempo

Che cosa è un Programma Intero?

Si consideri il **programma lineare**

$$(PL) \quad \begin{aligned} \max_{x_1, \dots, x_n} \quad & z(x_1, \dots, x_n) = \sum_{j=1}^n c_j x_j \\ \sum_{j=1}^n a_{ij} x_j &\leq b_i, \quad (i = 1, \dots, m) \\ x_1, \dots, x_n &\geq 0 \end{aligned} \quad \Longleftrightarrow \quad \begin{aligned} \max_x \quad & z(x) = c^T x \\ Ax &\leq b, \\ x &\geq 0, \end{aligned}$$

dove $x = (x_1, \dots, x_n)^T$, $c = (c_1, \dots, c_n)^T \in \mathbb{R}^n$, $A = (a_{ij}) \in M_{m \times n}$, $b = (b_1, \dots, b_m)^T \in \mathbb{R}^m$.

- Se tutte le variabili sono intere, si ha un **Programma (Lineare) Intero**:

$$(PI) \quad \max_{x \in \mathbb{R}^n} \{ z = c^T x \mid Ax \leq b, \underbrace{x \geq 0, x \in \mathbb{Z}^n}_{x \in \mathbb{Z}_+^n} \}$$

- Se ogni variabile può solo assumere i valori 0 o 1, cioè tutte le variabili sono binarie, si ha un **Programma Intero Binario**, o **Programma Intero 0-1**:

$$(PIB) \quad \max_{x \in \mathbb{R}^n} \{ z = c^T x \mid Ax \leq b, x \in \{0, 1\}^n \}$$

Che cosa è un Programma Intero?

Più in generale, se in un programma lineare *qualcuna delle variabili deve essere intera*, si ha un **Programma Intero Misto** (questa è la forma più generale):

$$\begin{aligned} \max_{\{x_i\}, \{y_k\}} \quad & z = \sum_{j=1}^n c_j x_j + \sum_{j=1}^p h_j y_j \\ \text{(PIM)} \quad & \sum_{j=1}^n a_{ij} x_j + \sum_{j=1}^p g_{ij} y_j \leq b_i, \quad (i = 1, \dots, m) \\ & x_1, \dots, x_n \geq 0, \\ & y_1, \dots, y_p \geq 0, y_1, \dots, y_p \in \mathbb{Z} \end{aligned} \quad \circ \quad \begin{aligned} \max_{x, y} \quad & z = c^T x + h^T y \\ & Ax + Gy \leq b, \\ & x \geq 0, y \geq 0, y \in \mathbb{Z}^p \end{aligned}$$

dove $x = (x_1, \dots, x_n)^T$, $A = (a_{ij}) \in M_{m \times n}$, $G = (g_{ij}) \in M_{m \times p}$, $b = (b_1, \dots, b_m)^T$, $c = (c_1, \dots, c_n)^T$, $h = (h_1, \dots, h_p)^T$.

- Il problema precedente è di **massimizzazione**; la funzione obiettivo può anche essere

$$\min z = c^T x + h^T y.$$

- Il problema precedente ha solo vincoli di "minore o uguale"; si può anche avere

- $\sum_{j=1}^n a_{ij} x_j + \sum_{j=1}^p g_{ij} y_j \geq b_i$, per qualche i , e/o

- $\sum_{j=1}^n a_{ij} x_j + \sum_{j=1}^p g_{ij} y_j = b_i$, per qualche i .

- Nel problema precedente le variabili sono soggette al vincolo di non negatività; le variabili possono anche non essere soggette a tale vincolo.

Formulare un problema come problema di ottimizzazione

Un problema di ottimizzazione è un modello matematico di un problema reale.

Per modellizzare un problema come problema di ottimizzazione, si seguono tipicamente i passi seguenti:

1. Definire quelle che sembrano le **variabili** necessarie; usualmente, espressioni quantitative delle decisioni da prendere.
2. Usare queste variabili per definire un insieme di **vincoli**; i valori assunti dalle variabili, compatibili con i vincoli, devono corrispondere alle soluzioni ammissibili del problema, e viceversa.
3. Usare queste variabili per definire la **funzione obiettivo** da massimizzare o minimizzare.

È bene ricordare che la definizione originaria delle variabili può essere rivista, e che spesso c'è più di un modo per modellizzare un problema.

Nel corso di Ottimizzazione Lineare abbiamo visto molti problemi che possono essere modellizzati tramite PL; illustreremo alcuni esempi di PL.

Problemi di Ottimizzazione Combinatoria

Molti problemi reali si presentano come Problemi di Ottimizzazione Combinatoria.

Dati:

- ▶ $N = \{1, \dots, n\}$, *insieme finito*,
- ▶ $c : N \rightarrow \mathbb{R}$, *funzione "lunghezza"*, tale che, per ogni $j \in N$, $c(j) = c_j \in \mathbb{R}$,
- ▶ $U = \mathcal{P}(N) = 2^N$, *insieme ambiente*; l'insieme U è l'insieme delle parti di N , cioè l'insieme costituito da tutti i possibili $2^{|N|}$ sottoinsiemi di N ,
- ▶ $X \subseteq U$, *insieme ammissibile*; X è una famiglia di sottoinsiemi S di N ,
- ▶ $z : X \subset U \rightarrow \mathbb{R}$, *funzione obiettivo*, definita da $z(S) = \sum_{j \in S} c_j$, per ogni $S \in X$.

Un *Problema di Ottimizzazione Combinatoria* è definito da:

$$(\text{POC}) \quad \max_{S \subseteq N} \left\{ z(S) = \sum_{j \in S} c_j : S \in X \right\} \quad (\text{o} \quad \min_{S \subseteq N} \left\{ z(S) = \sum_{j \in S} c_j : S \in X \right\})$$

Problemi di Ottimizzazione Combinatoria: Esempi

- **Problema MST:** in questo caso, se $G = (V, A)$, $N = A$, U è costituito da tutti i sottoinsiemi di archi di A e l'insieme ammissibile è $X = \{S \subseteq A : S \text{ è un albero ricoprente di } G\}$; il problema MST è espresso da $\min_{S \subseteq A} \{ \sum_{(i,j) \in S} \ell_{ij} : S \in X \}$.
- **Problema di cammino minimo da s a t :** se $G = (V, A)$, $s, t \in V$, $N = A$, U è costituito da tutti i sottoinsiemi di archi di A e l'insieme ammissibile è $X = \{S \subseteq A : S \text{ è un cammino da } s \text{ a } t\}$; il problema di cammino minimo è espresso da $\min_{S \subseteq A} \{ \sum_{(i,j) \in S} \ell_{ij} : S \in X \}$.
- **Problema di assegnazione:** ad ognuna di n persone si deve assegnare esattamente uno di n lavori, e viceversa; il costo per assegnare la persona i al lavoro j è c_{ij} . Il problema consiste nel trovare una assegnazione di costo minimo. In questo caso,
 - $N = \{(i, j) : i, j = 1, \dots, n\}$,
 - $U = \mathcal{P}(N)$ è costituito da tutti i sottoinsiemi di N
 - $X = \{S \subseteq N : S \text{ è una assegnazione}\} \subset U$;
 - il problema di assegnazione è espresso da $\min_{S \subseteq N} \{z(S) = \sum_{(i,j) \in S} c_{ij} : S \in X\}$.

Una **assegnazione** è descritta da un sottoinsieme S di N della forma

$$S = \{(1, j_1), (2, j_2), \dots, (n, j_n)\},$$

dove $j_k \neq j_m$, per ogni $k \neq m$, $k, m \in \{1, \dots, n\}$. Quindi S definisce una **permutazione** su $\{1, \dots, n\}$ e l'insieme ammissibile X è identificato con l'insieme Σ_n di tutte le permutazioni su $\{1, \dots, n\}$. Si ha che $|X| = |\Sigma_n| = n!$. (In particolare, $\forall \sigma \in \Sigma_n$, $z(\sigma) = \sum_{i=1}^n c_{i\sigma(i)}$.)

Se $n = 3$, per esempio, $X = \{S_1, S_2, S_3, S_4, S_5, S_6\}$, dove $S_1 = \{(1, 1), (2, 2), (3, 3)\}$, $S_2 = \{(1, 2), (2, 3), (3, 1)\}$, $S_3 = \{(1, 3), (2, 2), (3, 1)\}$, $S_4 = \{(1, 1), (2, 3), (3, 2)\}$, $S_5 = \{(1, 2), (2, 1), (3, 3)\}$, $S_6 = \{(1, 3), (2, 1), (3, 2)\}$.

Ottimizzazione Combinatoria

Proprietà fondamentali dei problemi di Ottimizzazione Combinatoria (OC):

1. Tutti i problemi di OC sono definiti su insiemi ammissibili finiti;
2. La funzione obiettivo può essere calcolata sugli elementi dell'insieme ammissibile.

Esiste quindi un algoritmo “universale” per i problemi di OC:

- ▶ enumerare tutte le soluzioni ammissibili e confrontare i valori assunti dalla funzione obiettivo su tutte le soluzioni ammissibili,
- ▶ tale algoritmo si chiama ENUMERAZIONE TOTALE.

Purtroppo, nella maggior parte dei casi, il numero di soluzioni ammissibili è talmente grande che la procedura di ENUMERAZIONE TOTALE risulta **impraticabile**.

Difficoltà computazionali della ENUMERAZIONE TOTALE

Le difficoltà computazionali della Enumerazione Totale associata al problema di assegnazione (e in generale a ogni problema di Ottimizzazione Combinatoria) sono ben illustrate da G. B. Dantzig in un articolo del 2002,¹ del quale riportiamo testualmente il seguente estratto:

A simple example may serve to illustrate the fundamental difficulty of finding a solution to a planning problem once it is formulated. Consider the problem of assigning 70 men to 70 jobs. Suppose a value or benefit v_{ij} would result if the i th man is assigned to the j th job. An activity consists in assigning the i th man to the j th job. The restriction are: (i) each man must be assigned a job (there are 70 such), and (ii) each job must be filled (also 70). The level of an activity is either 1, meaning it will be used, or 0, meaning it will not. Thus there are 2×70 or 140 restrictions, 70×70 or 4900 activities with 4900 corresponding 0-1 decision variables x_{ij} . Unfortunately there are $70!$ different possible solutions or ways to make the assignments x_{ij} . The problem is to compare the $70!$ solutions with one another and to select the one which results in the largest sum of benefits from the assignments. Now $70!$ is a big number, greater than 10^{100} . Suppose we had a computer capable of doing a million (10^6) calculations per second available at the time of the big bang fifteen billion years ago. Would it have been able to look at all the $70!$ combinations by the year 1990? The answer is no! Suppose instead it could perform at nanosecond (10^{-9} s) speed and make 1 billion (10^9) complete assignments per second? The answer is still no. Even if the Earth were filled with such computers all working in parallel, the answer would still be negative. If, however, there were 10^{40} Earths circling the sun each filled solid with nanosecond speed computers all programmed in parallel from the time of the big bang until the Sun grows cold, then perhaps the answer might be, yes.

¹G. B. Dantzig, Linear Programming, *Operations Research* **50** (2002), no. 1, 42–47.

Ottimizzazione Combinatoria e Variabili Binarie

- ▶ L'insieme ammissibile di un problema di Ottimizzazione Combinatoria è definito da un sottoinsieme X dell'insieme delle parti $\mathcal{P}(N) = 2^N$ di un insieme finito $N = \{1, \dots, n\}$.
- ▶ La funzione obiettivo di un problema di Ottimizzazione Combinatoria è definita da $z(S) = \sum_{j \in S} c(j)$, dove $S \in X$ e $c : N \rightarrow \mathbb{R}$ è una qualche funzione "lunghezza".

Una soluzione ammissibile $S \in X$ può essere descritta da un vettore binario $x \in \{0, 1\}^n$.

- ▶ A questo proposito, si introduce una **variabile binaria** x_j , per ogni $j \in N$.
- ▶ Una soluzione ammissibile $S \in X$ può essere rappresentata da un **vettore binario** $x \in \{0, 1\}^n$, se si pone

$$x_j := \begin{cases} 1, & \text{se } j \in S \\ 0, & \text{altrimenti} \end{cases}.$$

- ▶ In base a questa definizione, $S = \{j \in N : x_j = 1\}$. È quindi equivalente pensare alle soluzioni ammissibili sia come a sottoinsiemi S di N sia come a vettori binari x di $\mathbb{R}^n$.
- ▶ Pertanto, **l'insieme ammissibile X può essere rappresentato da un sottoinsieme di $\{0, 1\}^n$.**
- ▶ Oltre ai problemi di cammino minimo, dell'albero ricoprente di costo minimo (MST) e di assegnazione, le considerazioni precedenti si applicano, fra gli altri, al problema dello zaino, di copertura e del commesso viaggiatore, problemi che saranno discussi nel seguito.

Programmi Interi Binari (PIB) - Problema dello Zaino 0-1

Si dispone di un *budget* b da investire in n progetti durante il prossimo anno; a_j indica l'investimento richiesto per finanziare il progetto j , mentre c_j è il guadagno atteso se si intraprende il progetto j , $j = 1, \dots, n$. L'obiettivo è quello di finanziare un insieme di progetti in modo che sia massimizzato il guadagno atteso senza eccedere il *budget* b .

Variabili: $x_j := \begin{cases} 1, & \text{se } j \text{ viene scelto} \\ 0, & \text{altrimenti} \end{cases}, j = 1, \dots, n.$

Formulazione:

$$\begin{aligned} \max \quad & z(x) = \sum_{j=1}^n c_j x_j && \text{(Guadagno atteso)} \\ & \sum_{j=1}^n a_j x_j \leq b, && \text{(Non si può superare il budget)} \\ & x_j \in \{0, 1\}, j = 1, \dots, n && \text{(Tutte le variabili sono binarie)} \end{aligned}$$

Osservazione

Il Problema dello Zaino è un problema di ottimizzazione combinatoria: N è l'insieme degli n progetti, $U = \mathcal{P}(N)$, e $X = \{S \subset N : \sum_{j \in S} a_j \leq b\}$ è rappresentato da $\{(x_1, \dots, x_n)^T \in \{0, 1\}^n : \sum_{j=1}^n a_j x_j \leq b\}$. $|U| = 2^n$. Se $b = (\sum_{j=1}^n a_j)/2$, almeno la metà dei sottoinsiemi di N sono ammissibili, e quindi X ha almeno 2^{n-1} elementi.

Programmi Interi Binari (PIB) - Problema di Assegnazione

Ci sono n persone per svolgere n lavori, ognuno dei quali deve essere completato. Ogni persona deve svolgere esattamente un lavoro, e ogni lavoro deve essere svolto da esattamente una persona. Il costo stimato per assegnare la persona i al lavoro j è c_{ij} , $i, j = 1, \dots, n$. Il problema consiste nel trovare un'assegnazione di costo minimo.

Variabili: $x_{ij} := \begin{cases} 1, & \text{se la persona } i \text{ svolge il lavoro } j \\ 0, & \text{altrimenti} \end{cases} \quad (i, j = 1, \dots, n).$

Formulazione:

$$\begin{aligned} \min \quad & z(x) = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} && \text{(Costo dell'assegnazione)} \\ & \sum_{j=1}^n x_{ij} = 1 \quad (i = 1, \dots, n) && \text{(Ogni persona svolge un lavoro)} \\ & \sum_{i=1}^n x_{ij} = 1 \quad (j = 1, \dots, n) && \text{(Ogni lavoro è svolto da una persona)} \\ & x_{ij} \in \{0, 1\}, \quad i, j = 1, \dots, n && \text{(Tutte le variabili sono binarie)} \end{aligned}$$

Osservazione

Il Problema di Assegnazione è un problema di OC: $N = \{(i, j) : i, j = 1, \dots, n\}$, $U = \mathcal{P}(N)$ e X è l'insieme delle assegnazioni. Un elemento $S \in X$ è un sottoinsieme di N della forma

$$S = \{(1, j_1), (2, j_2), \dots, (n, j_n)\},$$

dove $j_k \neq j_m$, per ogni $k \neq m$, $k, m \in \{1, \dots, n\}$. Ogni S definisce una **permutazione** su $\{1, \dots, n\}$ e X coincide con l'insieme delle permutazioni su $\{1, \dots, n\}$. $|X| = n!$.

Programmi Interi Binari (PIB) - Problema di Copertura

Il Pronto Soccorso di un ospedale deve selezionare un gruppo di medici per il servizio di reperibilità in modo che ogni possibile intervento medico si renda necessario possa essere effettuato da almeno uno dei medici scelti per il servizio (i possibili interventi sono elencati in una lista ufficiale).

Conoscendo il salario addizionale da pagare ad ogni singolo medico disponibile al servizio, nonché gli interventi che ogni singolo medico può effettuare, l'obiettivo è quello di scegliere i medici in modo da minimizzare il costo e coprire ogni intervento.

Esempio

	Med 1	Med 2	Med 3	Med 4	Med 5	Med 6	Med 7
Intervento 1	*			*		*	
Intervento 2	*				*	*	
Intervento 3		*	*				
Intervento 4	*				*	*	
Intervento 5		*	*				*
Intervento 6		*			*		

Formulazione del Problema di Copertura come PIB

Rappresentazione dei dati: matrice di incidenza.

Con m interventi possibili e n medici disponibili per il servizio, i dati possono essere rappresentati da una matrice $A = (a_{ij}) \in M_{m \times n}$ (**matrice di incidenza**), dove

$$a_{ij} := \begin{cases} 1, & \text{se l'intervento } i \text{ è effettuato dal medico } j \\ 0, & \text{altrimenti} \end{cases} \quad (i = 1, \dots, m; j = 1, \dots, n).$$

Inoltre, sia c_j il salario aggiuntivo da pagare al medico j , $j = 1, \dots, n$.

Variabili: $x_j := \begin{cases} 1, & \text{se il medico } j \text{ è reperibile} \\ 0, & \text{altrimenti} \end{cases} \quad (j = 1, \dots, n).$

Formulazione:

$$\begin{array}{ll} \min & z(x) = \sum_{j=1}^n c_j x_j \quad (\text{Costo salari pagati}) \\ & \sum_{j=1}^n a_{ij} x_j \geq 1 \quad (i = 1, \dots, m) \quad (\text{Almeno un medico effettua l'intervento } i) \\ & x_j \in \{0, 1\}, j = 1, \dots, n \quad (\text{Tutte le variabili sono binarie}) \end{array}$$

ovvero

$$\begin{array}{ll} \min & z(x) = c^T x \\ & Ax \geq e, \\ & x \in \{0, 1\}^n, \end{array}$$

dove $x = (x_1, \dots, x_n)^T$, $c = (c_1, \dots, c_n)^T \in \mathbb{R}^n$, $e = (1, \dots, 1)^T \in \mathbb{R}^m$.

Problema di Copertura (Set Covering Problem): definizione formale

Dati del problema:

- ▶ $N = \{1, \dots, n\}$, insieme finito
- ▶ $M = \{1, \dots, m\}$, insieme finito
- ▶ Un sottoinsieme $M_j \subset M$, per ogni $j \in N$
- ▶ Un “peso” (costo, lunghezza, ecc.) $c_j \in \mathbb{R}$, per ogni $j \in N$

Definizione

Una **copertura** di M (tramite N) è un sottoinsieme $T \subseteq N$ tale che $\bigcup_{j \in T} M_j = M$.

Nell'esempio precedente: N è l'insieme dei medici disponibili, M è l'insieme di tutti i possibili interventi, M_j è l'insieme degli interventi che il medico j può effettuare, c_j è il salario aggiuntivo del medico j . T è l'insieme dei medici scelti per il servizio. T deve essere una copertura di M .

Nel problema di copertura, si cerca una copertura di costo minimo:

$$\min_T \left\{ \sum_{j \in T} c_j : T \text{ è una copertura} \right\}$$

Osservazione

Il problema di copertura è un problema di ottimizzazione combinatoria.

Problema di Copertura: formulazione

La **matrice di incidenza** del problema (che è desunta dai dati!) è $A = (a_{ij})$, dove

$$(1) \quad a_{ij} := \begin{cases} 1, & \text{se } i \in M_j \\ 0, & \text{altrimenti} \end{cases} \quad (\forall i \in M; \forall j \in N).$$

Variabili: $x_j := \begin{cases} 1, & \text{se } j \in T \\ 0, & \text{altrimenti} \end{cases} \quad (\forall j \in N).$

Vincoli: $\sum_{j=1}^n a_{ij} x_j \geq 1 \quad (\forall i \in M).$

Funzione obiettivo: $z = \sum_{j=1}^n c_j x_j.$

Formulazione del Problema di Copertura (Set Covering Problem):

$$\begin{array}{ll} \min & z(x) = \sum_{j=1}^n c_j x_j \\ & \sum_{j=1}^n a_{ij} x_j \geq 1 \quad (\forall i \in M) \\ & x_j \in \{0, 1\}, j = 1, \dots, n \end{array} \quad \Longleftrightarrow \quad \begin{array}{ll} \min & z(x) = c^T x \\ & Ax \geq e, \\ & x \in \{0, 1\}^n, \end{array}$$

$A = (a_{ij})$, matrice di incidenza, $c = (c_1, \dots, c_n)^T \in \mathbb{R}^n$, $e = (1, \dots, 1)^T \in \mathbb{R}^m$.

Problema correlato: Impaccamento (Set Packing)

Stessi dati del Problema di Copertura:

- ▶ $N = \{1, \dots, n\}$, insieme finito
- ▶ $M = \{1, \dots, m\}$, insieme finito
- ▶ Un sottoinsieme $M_j \subset M$, per ogni $j \in N$
- ▶ Un "peso" (costo, lunghezza, ecc.) $c_j \in \mathbb{R}$, per ogni $j \in N$

Definizione

Un **impaccamento** di M (tramite N) è un sottoinsieme $T \subseteq N$ tale che $M_j \cap M_k = \emptyset$, per ogni $j, k \in T, j \neq k$.

Nel problema di impaccamento, si cerca un impaccamento di peso massimo:

$$\max_T \left\{ \sum_{j \in T} c_j : T \text{ è un impaccamento} \right\}$$

Formulazione del Problema di Impaccamento (Set Packing Problem):

$$\begin{array}{ll} \max & z(x) = \sum_{j=1}^n c_j x_j \\ & \sum_{j=1}^n a_{ij} x_j \leq 1 \quad (\forall i \in M) \\ & x_j \in \{0, 1\}, j = 1, \dots, n \end{array} \quad \Longleftrightarrow \quad \begin{array}{ll} \max & z(x) = c^T x \\ & Ax \leq e, \\ & x \in \{0, 1\}^n, \end{array}$$

dove $A = (a_{ij})$ è la **matrice di incidenza**, definita come in (1) a partire dai dati del problema, $c = (c_1, \dots, c_n)^T \in \mathbb{R}^n$, $e = (1, \dots, 1)^T \in \mathbb{R}^m$.

Altro problema correlato: Partizione (Set Partition)

Stessi dati del Problema di Copertura e di Impaccamento:

- ▶ $N = \{1, \dots, n\}$, insieme finito
- ▶ $M = \{1, \dots, m\}$, insieme finito
- ▶ Un sottoinsieme $M_j \subset M$, per ogni $j \in N$
- ▶ Un "peso" (costo, lunghezza, ecc.) $c_j \in \mathbb{R}$, per ogni $j \in N$

Definizione

Una **partizione** di M (tramite N) è un sottoinsieme $T \subseteq N$ che è al contempo una copertura e un impaccamento, cioè $\bigcup_{j \in T} M_j = M$ e $M_j \cap M_k = \emptyset$, per ogni $j, k \in T, j \neq k$.

Nel problema di partizione, si cerca una partizione di peso massimo (o minimo):

$$\max_T \left\{ \sum_{j \in T} c_j : T \text{ è una partizione} \right\}$$

Formulazione del Problema di Partizione (Set Partition Problem):

$$\begin{array}{ll} \max & z(x) = \sum_{j=1}^n c_j x_j \\ & \sum_{j=1}^n a_{ij} x_j = 1 \quad (\forall i \in M) \\ & x_j \in \{0, 1\}, j = 1, \dots, n \end{array} \quad \Longleftrightarrow \quad \begin{array}{ll} \max & z(x) = c^T x \\ & Ax = e, \\ & x \in \{0, 1\}^n, \end{array}$$

dove $A = (a_{ij})$ è la **matrice di incidenza**, definita come in (1) a partire dai dati del problema, $c = (c_1, \dots, c_n)^T \in \mathbb{R}^n$, $e = (1, \dots, 1)^T \in \mathbb{R}^m$.